Raspberry Pi 4 B — running one test on the board, step by step

This page runs one µOS++ test on a Raspberry Pi 4 B (BCM2711, 4× Cortex-A72) through OpenOCD, by hand, for the 64-bit (AArch64) port. It does exactly what the test scripts do (`micro-os-plus-iii-aarch64/test/boards/rpi4b/hw.sh`, which calls `micro-os-plus-iii/test_smpl/run-hw.sh`), written out as plain commands: reset, load, run, watch, end.

The probe is a Raspberry Pi **debugprobe** (a Pico) used as a **CMSIS-DAP v2** adapter in **JTAG** mode, through OpenOCD (xPack 0.12.0-7.1). The Pico runs an updated firmware with optimized CMSIS-DAP/JTAG code (it still reports `FW Version = 2.0.0`), which loads about 38× faster than the stock debugprobe firmware (see “Load speed” below).

What happens, in short

1. **Reset.** OpenOCD connects, halts core 0, and writes the BCM2711 PM watchdog (`0xfe100024` = `0x5a000001`, then `0xfe10001c` = `0x5a000020`). The whole SoC resets and boots again from the SD card. OpenOCD loses the link and exits.
2. **Wait 15 s** for the firmware to boot and switch the JTAG pins on again. Core 0 runs the firmware stub; cores 1–3 wait in the stub’s spin-table loop.
3. **Connect and halt core 0** (it stops at EL2, MMU off). Cores 1–3 are not halted (see step 6).
4. **Semihosting on**, on all four cores.
5. **Load.** `load_image` copies the test’s ELF into RAM at `0x80000`.
6. **Run.** Core 0’s PC is set to `0x80000` (`_start`) and core 0 is resumed. Cores 1–3 go on waiting in the firmware’s loop until the kernel writes their spin-table slot.
7. **Watch.** The test’s console is the UART. The verdict line (`RESULT: PASS|FAIL|SKIP`) also goes out through semihosting, where the runner sees it.
8. **End.** As soon as the verdict appears, the runner stops OpenOCD. The next test starts again at step 1; no power cycle is needed.

The hardware

What	Where
JTAG	the Pi’s GPIO 22–27 (ALT4: TRST, RTCK, TDO, TCK, TDI, TMS), enabled by `enable_jtag_gpio=1` in `config.txt`
Probe	Pico debugprobe, USB `2e8a:000c`, CMSIS-DAP v2, firmware 2.0.0
Console	the Pi’s PL011 UART (GPIO 14/15), 115200 8N1, here on a CH340 adapter at `/dev/ttyUSB0`
SD card	the boot card: firmware, `config.txt` and a `kernel8.img`. The test’s `config.txt` is `micro-os-plus-iii-aarch64/test/boards/rpi4b/config.txt`

The hwd tests that use the SD card (`sd_test`, `smp-mat-sdcard-test`, `smp-num-test`, `smp-pipeline-test`) mount the card’s **existing FAT32 boot partition** and write only under a `tests/` folder. They never format the card and never touch the boot files.

Step 0 — set the paths

```
W=/home/dan/Desktop/micro-os-plu-iii-smp-pr-github
OCD_DIR=$HOME/.local/xPacks/@xpack-dev-tools/openocd/0.12.0-7.1/.content
OCD=$OCD_DIR/bin/openocd
SCRIPTS=$OCD_DIR/openocd/scripts
BOARD=$W/micro-os-plu-iii-aarch64/test/boards/rpi4b
CFG=$BOARD/openocd-cmsis-dap-rpi4b.cfg
C=aarch64-rpi4b-cmake-gcc-debug
TESTS=$W/micro-os-plus-iii/tests
ELF_DIR=$TESTS/build/$C/platform-bin/port-tests/test
```

Step 1 — build the tests

```
cd $TESTS
xpm run install --config $C     # first time only
xpm run prepare --config $C
xpm run build   --config $C
```

Each test gives `<app>-hwd` (for the board) and `<app>-qemu`. The `-hwd` name is a symlink to `<app>-hwd.elf`.

Step 2 — choose the test

```
ls $ELF_DIR/*-hwd
APP=smp-mat-test
ELF=$ELF_DIR/$APP-hwd
```

What matters for the load time is the size of the loadable segment (`FileSiz` of the `LOAD` line), not the size of the ELF file (that one also holds the debug information, which is never sent to the board):

```
$TESTS/build/$C/xpacks/.bin/aarch64-none-elf-readelf -lW $ELF | grep LOAD
```

With the optimized probe firmware the load runs at about **39.5 KB/s** (smp_test4: 70,296 bytes in 1.78 s, measured). The stock debugprobe firmware did about 1 KB/s (see “Load speed” below). The new column is that rate applied to each test’s size; only smp_test4 was timed:

Test	Loaded bytes	Load (optimized firmware)	Load (stock firmware)
cmsis-os-validator	151,640	~3.8 s	~2.4 min
mutex-stress	94,488	~2.4 s	~1.5 min
rtos-apis	228,136	~5.8 s	~3.5 min
sd_test	83,512	~2.1 s	~1.3 min
smp-mat-sdcard-test	93,736	~2.4 s	~1.5 min
smp-mat-test	77,608	~2.0 s	~1.2 min
smp-num-test	91,160	~2.3 s	~1.4 min
smp-pipeline-test	146,472	~3.7 s	~2.3 min
smp-pro-cons-test	88,968	~2.3 s	~1.4 min
smp_test0 … smp_test4	65,656 … 74,744		~1.1 min

Test	Loaded bytes	Load (optimized firmware)	Load (stock firmware)
		~1.7 … 1.9 s (smp_test4: 1.78 s measured)	

Step 3 — open the serial console

In a second terminal, and leave it open:

```
tio -b 115200 /dev/ttyUSB0
```

The test's whole output appears here once it is started (step 6). On the board, only the verdict lines go through semihosting.

Step 4 — reset the board

```
pgrep -x openocd && echo "an OpenOCD still holds the probe: pkill -x openocd"

$OCD -s $BOARD -s $SCRIPTS -f $CFG -c init \
  -c "targets bcm2711.cpu0; halt; \
      catch {mww 0xfe100024 0x5a000001}; \
      catch {mww 0xfe10001c 0x5a000020}; \
      shutdown"
```

`0x5a` is the PM password; `0xfe100024` (PM_WDOG) gets a timeout of 1 tick, and `0xfe10001c` (PM_RSTC) gets "full reset".

The address is the **virtual** one, not `mww phys`. With the MMU and D-cache on (a test was running), `mww phys` makes OpenOCD clean the caches by set/way over JTAG first, which never finishes at this link speed.

What a good reset prints: the TAP is found, and then the link drops when the SoC resets:

```
Info : JTAG tap: bcm2711.cpu tap/device found: 0x4ba00477 (mfg: 0x23b (ARM Ltd), part: 0xba00,
ver: 0x4)
...
Error: Invalid ACK (0) in DAP response
Error: [bcm2711.cpu0] Polling failed, trying to reexamine
```

Those last errors are the proof that the reset happened. If the output has no `tap/device found` line, or a Tcl `Traceback`, the watchdog was **not** written: fix that first. The runner checks this and stops with `watchdog reset FAILED`.

The same reset is in `$BOARD/reset.cfg`:

```
$OCD -s $BOARD -s $SCRIPTS -f $BOARD/reset.cfg
```

Step 5 — wait for the board to boot

```
sleep 15
```

Step 6 — load the test and start it

Write the session's commands to a file (this is what the runner generates, in `$ELF_DIR/.hw-logs/<app>.cfg`):

```
cat > /tmp/run-$APP.cfg <<CFG
echo "--- stage: halt cores 0 ---"
targets bcm2711.cpu0; halt

echo "--- stage: enable semihosting on cores 0 1 2 3 ---"
# Per-target: a core whose semihosting is off stalls at a debug halt the first
# time it traps, so every core needs it, not just core 0.
foreach core {0 1 2 3} { targets [format {bcm2711.cpu%d} \$core]; arm semihosting enable }
if {[info procs rpi4b_enable_hde] ne ""} { rpi4b_enable_hde }

echo "--- stage: load_image $ELF ---"
targets bcm2711.cpu0
load_image $ELF

echo "--- stage: resume cores 0 at 0x80000 ---"
targets bcm2711.cpu0
reg pc 0x80000
echo "  start PC core 0 = [reg pc]"
resume
CFG
```

Only **core 0** is halted, loaded through and resumed. Cores 1–3 are never halted: they keep running in the firmware's spin loop until the kernel writes their spin-table slot. Halting them too looks harmless but is not: a resume of core 0 restarts the other cores through the CTI (cross-trigger), so OpenOCD never resumes them itself, and a core it halted at the start later halts with the registers OpenOCD cached at that first halt. OpenOCD then cannot serve that core's semihosting call, the core stays halted, and the test stalls.

`0x80000` is `_start`, the ELF's entry point (`aarch64-none-elf-readelf -h $ELF | grep Entry`).

Then run OpenOCD with it, and **leave it running**: it serves the test's semihosting calls.

```
$OCD -s $BOARD -s $SCRIPTS -f $CFG -c init -f /tmp/run-$APP.cfg 2>&1 | tee /tmp/run-$APP.log
```

What it prints before the load, all normal:

```
Info : JTAG tap: bcm2711.cpu tap/device found: 0x4ba00477 ...
Error: JTAG-DP STICKY ERROR
Error: Failed to read memory at 0x80410088        (and 0x80510088, 0x80610088, 0x80710088)
bcm2711.cpu0 halted in AArch64 state due to debug-request, current mode: EL2H
cpsr: 0x000003c9 pc: 0x200
```

The `Failed to read memory at 0x80n10088` lines

One pair of `JTAG-DP STICKY ERROR` / `Failed to read memory` lines per core, always the same four addresses. They are harmless: the test is not affected, and the runner ignores exactly these (`UOS_HW_IGNORE_ERRORS` in `hw.sh`).

Address	Core	Register
0x80410088	0	EDSCR (debug base 0x80410000 + 0x088)
0x80510088	1	EDSCR (0x80510000 + 0x088)

Address	Core	Register
0x80610088	2	EDSCR (0x80610000 + 0x088)
0x80710088	3	EDSCR (0x80710000 + 0x088)

EDSCR is the core's External Debug Status and Control Register. What happens, for each core, in this order (OpenOCD src/target/target.c, target_examine_one):

1. OpenOCD fires the core's examine-start event. The config hooks rpi4b_unwedge <debug base> there, whose first action is to read EDSCR through the APB-AP.
2. Only then does OpenOCD examine the core (aarch64_examine_first), and the first thing that does is write OSLAR = 0: it unlocks the core's debug **OS Lock**.

The board has just been reset by the watchdog (Step 4), and a reset sets the OS Lock of every core (OpenOCD reports it as external reset detected). While the OS Lock is set, the core refuses an external read of EDSCR; the JTAG-DP flags the refused access as a sticky error, and OpenOCD prints the two lines.

Why it does no harm:

- rpi4b_unwedge reads EDSCR inside catch: the failed read makes it return, and the session goes on. After a fresh reset there is nothing to unwedge anyway.
- The same EDSCR read at examine-end (rpi4b_enable_core_hde), after OpenOCD has unlocked the core, gives no error – the proof that the OS Lock is the cause.
- rpi4b_unwedge matters for the reset session of Step 4, which connects to a board that has *not* been reset yet: there the OS Lock is clear and a core left stuck by a killed session can be recovered.

To silence them, rpi4b_unwedge would have to write OSLAR = 0 (<debug base> + 0x300) itself before its first read – the same write OpenOCD does. The config does not do that today; the lines are only cosmetic.

What the config file $CFG sets up, besides the board:

Line	Why
source [find interface/cmsis-dap.cfg], transport select jtag	the probe, in JTAG mode
source [find board/rpi4b.cfg]	OpenOCD's BCM2711 target: bcm2711.cpu0..3, bcm2711.dap, bcm2711.ap (not grouped: USE_SMP is 0), linked by their CTIs
adapter speed 4000	4 MHz TCK
gdb port disabled	no GDB, OpenOCD only
rpi4b_unwedge on every examine-start	clears a core's stuck debug state (EDSCR ERR/MA, TX/RX full) left by an earlier session
bcm2711.dap memaccess 0 on every examine-end	no idle TCKs after each access: the load is 1.74× faster (see below). It has to be set after init, and every core's examine resets it to 10, so it is set again after each one

Step 7 — watch the result

On the UART (tio), the test's own output, ending with the verdict:

```
RESULT: PASS
```

In the OpenOCD output, only the verdict line arrives (through semihosting). The runner waits for one of:

In the OpenOCD output	Meaning
`RESULT: PASS`	passed
`RESULT: SKIP`	skipped
`RESULT: FAIL`, `[FAIL]`, `FAIL:`, `FATAL:`	failed
OpenOCD exits	OpenOCD died
`Invalid ACK`, `Polling failed`, `Examination failed`	debug link lost
any other `Error:` line	OpenOCD error
nothing within the time limit (1800 s)	timeout

Step 8 — end OpenOCD

When the verdict is there, stop OpenOCD (Ctrl-C, or):

```
pkill -x openocd
```

Stopping it while it is talking to the probe prints `Error: error handling USB events: System call interrupted (perhaps due to signal)`: that is the stop itself, not a problem.

The board is now running (or stopped in) the finished test. The next test starts again at step 4: the watchdog reset puts the board back in a clean state, so there is **no power cycle** between tests.

The same, with the scripts

One test, with its result in CTest (reset, wait, load, run, verdict, end):

```
cd $TESTS
export PATH=$PWD/build/$C/xpacks/.bin:$PATH
ctest --test-dir build/$C -R '^aarch64-rpi4b-smp-mat-test-hwd$' -V
```

Or through xpm, which builds first:

```
xpm run test-smp-mat-test-hwd --config $C
```

All hwd tests, one after another (each one resets the board first):

```
xpm run test-hwd --config $C          # prepare + build, then ctest -V -L hwd
```

The port's script directly, without CTest:

```
$BOARD/hw.sh list                     # what is built, with time limits
BUILD=$TESTS/build/$C/platform-bin/port-tests $BOARD/hw.sh smp-mat-test 1800
```

Files the runner leaves in `$ELF_DIR/.hw-logs/`:

File	What
`<app>.reset.log`	OpenOCD output of the reset session (step 4)
`<app>.cfg`	the commands of step 6
`<app>.log`	OpenOCD output of the test session (steps 6–7)

Settings (environment variables) the scripts take:

Variable	Default	What
`UOS_HW_RESET_WAIT`	15	seconds between the watchdog reset and the connect
`UOS_HW_CORES`	`0`	cores that are halted, loaded through and resumed
`UOS_HW_SEMIHOST_CORES`	`0 1 2 3`	cores with semihosting enabled (without halting them)
`UOS_HW_ADAPTER_KHZ`	(config: 4000)	JTAG clock, set after `init`
`UOS_HW_PRELOAD`	the two watchdog writes	the reset commands of step 4
`UOS_HW_IGNORE_ERRORS`	`JTAG-DP STICKY ERROR|Failed to read memory at 0x80[4-7] [0-9a-f]+$`	`Error:` lines that do not end the run
`OPENOCD`	newest xPack OpenOCD	the OpenOCD binary

From VS Code

The VS Code project (`RPI4B/64/rpi4b`) uses the same steps with its own config, `rpi4b-cmsis-dap.cfg` (same `rpi4b_unwedge` and `memaccess 0` events):

```
./reset-board.sh 30      # step 4 + a 30 s wait (step 5)
```

and then the “Debug Pi 4B” launch does steps 6–7 through GDB (`monitor halt`, `load`, `monitor arm semihosting enable`).

Load speed

With the **optimized probe firmware** (the Pico reflashed with new optimized CMSIS-DAP/JTAG code), measured on this setup at `adapter speed 4000`, with `bcm2711.dap memaccess 0`: smp_test4 (70,296 loaded bytes) in **1780 ms**, about **39.5 KB/s** (38.6 KiB/s), timed around `load_image` alone in OpenOCD (`clock milliseconds` before and after). That is about **38×** the stock firmware’s ~1.05 KB/s, and the whole test ran and passed (29.6 s in CTest, the 15 s reset wait included).

Why the stock firmware was slow

Measured on this setup with the stock debugprobe firmware 2.0.0, with OpenOCD’s packet-level log (`-d4`):

- OpenOCD’s CMSIS-DAP driver, in **JTAG** mode, sends raw `DAP_JTAG_Sequence` commands and encodes every TMS bit and every idle clock as its own 2-byte entry. One 32-bit access (a 35-bit DR scan plus idle clocks) fills a whole 64-byte USB packet: 4 bytes of data per packet.
- Each packet waits for its answer: **8.4 ms** per round trip on average (full-speed USB, the probe behind a hub).
- So the TCK frequency hardly matters: about 0.6 KB/s at any clock.

With `bcm2711.dap memaccess 0` (10 idle clocks per access removed), a 4 KB load went from 6.7 s to 3.9 s (1.74×) and read back identical. The rest of the cost was the round trip, and that is what the optimized probe firmware removed (above).

If something goes wrong

Symptom	**Cause, fix**
`LIBUSB_ERROR_BUSY`, or the probe is not found	another OpenOCD holds it: `pkill -x openocd`
the reset prints `The 'bcm2711.dap memaccess' command must be used after 'init'`	`memaccess` written as a plain line in the config; it must be in the `examine-end` events
the reset session shows no `tap/device found`	no JTAG: the board is off, or the firmware has not enabled the pins (`enable_jtag_gpio=1`), or it is still booting (wait longer: `UOS_HW_RESET_WAIT=30`)
`Error: [bcm2711.cpu1] not halted` at the resume, then OpenOCD exits	a script that halted cores 1–3 and resumes them after core 0: the core 0 resume already restarted them through the CTI. Halt and resume core 0 only (`UOS_HW_CORES=0`)
the output stops, and a core 1–3 shows `halted ... due to breakpoint`, `pc: 0x80`, EL1, registers equal to its first halt	that core was halted at the start of the session: OpenOCD later reads its stale cached registers and cannot serve its semihosting call. Do not halt cores 1–3 (`UOS_HW_CORES=0`)
the run log shows `RESULT:` without `PASS`/`FAIL`	the verdict was printed in pieces; the board's `uart.hpp` must mirror the whole verdict line (`mirror_verdict`) or use a single `puts()`
nothing on the UART after the halt	the load is still running: a few seconds with the optimized probe firmware, minutes with the stock one (see the table in step 2)
`mww phys` never ends	use the virtual address (step 4)